reduce the boilerplate required for some features and to improve performance by relying on compile-time analysis instead of runtime reflection.

.NET 6 introduced several source generator implementations, such as a high-performance logging API, which I discuss in this post: http://mng.bz/Y1GA. Even the Razor compiler used to compile .cshtml files was rewritten to use source generators!

In .NET 7, many new source generators were added. One such generator is the regular-expression generator, which can improve performance of your `Regex` instances, such as the one in listing 14.6. In fact, if you're using an IDE like Visual Studio, you should see a code fix suggesting that you use the new pattern. After you apply the code fix, listing 14.6 should look like the following instead, which is functionally identical but will likely be faster:

```
partial class KebabCaseParameterTransformer : IOutboundParameterTransformer
{
    public string? TransformOutbound(object? value)
    {
        if (value is null) return null;

        return MyRegex().Replace(value.ToString(), "$1-$2").ToLower();
    }

    [GeneratedRegex("([a-z])([A-Z])")]
    private static partial Regex MyRegex();
}
```

If you'd like to know more about how this source generator works and how it can improve performance, see this post at http://mng.bz/GyEO. If you'd like to learn more about source generators, or even write your own, see my series on the process at http://mng.bz/zX4Q.

You can register the parameter transformer in your application with the `AddRazorPagesOptions()` extension method in Program.cs. This method is chained after the `AddRazorPages()` method and can be used to customize the conventions used by Razor Pages. The following listing shows how to register the kebab-case transformer. It also shows how to add an extra page route convention for a given Razor Page.

Listing 14.7 Registering a parameter transformer using `RazorPagesOptions`